Smart Complaint Management System

Software Architecture and Design Specification

Section 3 Architecture | Member 3 scope
Prepared by Mohammed Zahoor Mashahir | PES1UG24CS579
Team 8 | Version 0.2

3 Architecture

We select a layered modular monolith for SCMS. A single Flask application coordinates complaint workflows through separate authentication, complaint, routing, audit, feedback and query modules. SQLite provides durable storage. This structure supports the fixed university escalation hierarchy while keeping deployment and transaction management manageable for the mini-project.

This architecture provides the component boundaries and data ownership for the Section 4 design. The design supports Students, Faculty-in-Charge at L1, Grievance Committee at L2 and Dean at L3. It uses manual SLA review and in-app notifications. No university ERP, SSO, external email service or background SLA scheduler is required.

3 1 Architecture pattern and rationale

Presentation routes validate requests and invoke application services. ComplaintService coordinates use cases; RoutingPolicy owns level transitions; the Repository isolates SQLite access. Services share an explicit transaction context when a complaint change requires audit, assignment or notification records. The modules are logical components within one deployment unit, not independent network services.

Option	Fit for SCMS	Tradeoff
Layered modular monolith	Selected. Matches Flask and SQLite constraints and allows one atomic workflow transaction.	Modules can be maintained independently, but share deployment and database availability.
Microservices	Separate deployment and service-owned stores would add complexity for a small team.	Distributed writes, authentication and operation are disproportionate to this scope.
Unstructured single module	Simple initially, but duplicates role checks and routing rules.	Does not meet NFR-MAIN-01 separation of functionality.

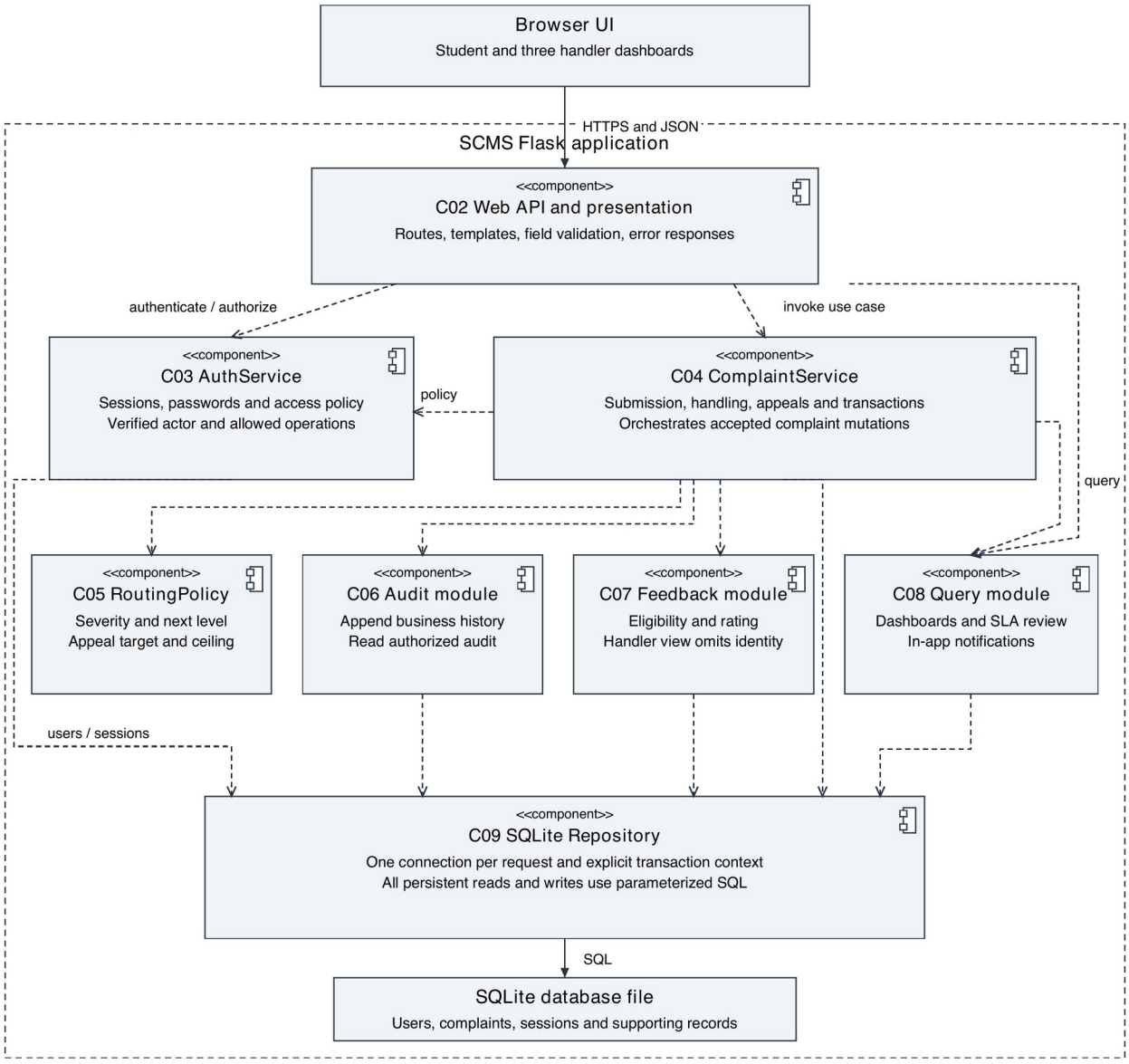
3 1 1 Technology and scope constraints

Python 3, Flask and SQLite are SRS constraints. Use a modern browser and a Windows or Linux host. Dependency versions are pinned when implementation begins. This specification describes proposed architecture; it does not report an implemented web application or passed acceptance tests.

3 2 UML component diagram

Figure AC01 shows the presentation, application and persistence layers. Dependencies point toward the required component. AuthService supplies verified identity and policy; application services enforce it on every protected use case.

AC01 SCMS UML Component Diagram



Dashed arrows show dependencies; solid arrows show external communication or SQL access.
C01 is the Browser UI. RoutingPolicy is pure and has no database dependency.

AC01. C01 is the Browser UI. C02 exposes the browser-facing interface. C04 owns workflow transactions. C06 to C08 access C09 using the same transaction when called during a mutation; read requests use their own request connection.

3 3 Component descriptions and interfaces

ID and component	Responsibility	Communicates with
C01 Browser UI	Registration, login, complaint forms, status and role-specific dashboards. Displays errors and permitted actions.	C02 over HTTPS; never C09 or the database.
C02 Web API and presentation	Flask routes and templates; request parsing, validation, authentication guard and consistent responses.	C03 identity; C04 use cases; C07 feedback; C08 queries.
C03 AuthService	Registration, password verification, session lifecycle and ownership/role/assignment access policy.	C09 users/sessions; verified actor supplied to C02 and service callers.
C04 ComplaintService	Coordinates submission, resolve, reject, escalation and appeal. Owns the transaction boundary.	C03 policy, C05 routing, C06 audit, C08 notices and C09 state.
C05 RoutingPolicy	Pure initialLevel, nextLevel and appealTarget decisions. Enforces L3 ceiling centrally.	C04 only; no persistent state.
C06 Audit module	Appends accepted business events and returns authorized chronological history.	C03 visibility; C09 audit and assignment history.
C07 Feedback module	Resolved-owner eligibility, rating validation, one feedback record and identity-suppressed handler view.	C03 policy; C09 complaints and feedback.
C08 Query module	Filtered dashboards, manual SLA calculation and notifications. SLA escalation delegates to C04.	C03 policy; C04 escalation; C09 read models/notices.
C09 SQLite Repository	Parameterized persistence, request connections, constraints and transaction support.	SQLite file; C03, C04, C06, C07 and C08.

3 3 1 Interface ownership

AuthService, ComplaintService, RoutingPolicy and SQLite Repository retain the interface names in Section 4. Audit and feedback are dedicated modules beneath those interfaces. Query methods support listVisible, audit reads, SLA review and notification reads. Every service receives a verified actor and checks resource policy; no caller treats UI visibility as authorization.

Repository mutation helpers accept the caller transaction. Audit and notification helpers do not open a second independent transaction during a complaint change. This avoids a complaint committing without its required history. Detailed HTTP payloads and error contracts remain in Section 4.

3 4 Runtime workflows and state rules

3 4 1 Submission and assignment

C02 authenticates the Student and validates category, description and severity. C04 asks C05 for the initial level: Low/Medium to L1 and High to L2. C09 inserts the unique complaint, timestamps and initial assignment; C06 records SUBMIT/ASSIGN; C08 creates the role-queue notice. A single successful commit precedes confirmation. Invalid input or failed persistence leaves no partial complaint or success message.

3 4 2 Handler actions and appeal

C03 checks the current assignment before normal handler mutations. C04 validates state, mandatory remarks/reason and expected_version. C05 permits L1 to L2 and L2 to L3; L3 has no next level. A rejection records rejected_level. An owning Student may appeal once while Rejected: L1 to L2, L2 to L3, or L3 final review at L3. A unique appeal constraint and conditional update protect against concurrent duplicate appeals.

Operation	Resulting state	Required invariant
Submit	Submitted at L1 or L2	Student owner, valid fields, persisted unique ID.
Resolve	Resolved	Current handler and nonblank remarks.
Reject	Rejected	Current handler, reason and rejecting level recorded.
Escalate	Escalated at next level	Current handler; L3 ceiling; no terminal-state action.
Appeal	Under Appeal Review	Owning Student; Rejected; not previously appealed.

3 4 3 Dashboards audit and feedback

Students see their own complaints. Faculty/Committee see their current queue. The Dean sees complaints that reached L3 and excludes earlier-level resolutions. Owners and handler levels with current or historic assignment can read audit history; historic visibility grants no mutation right. C07 prompts the owner for feedback on a resolved complaint without existing feedback and omits identity from the handler projection.

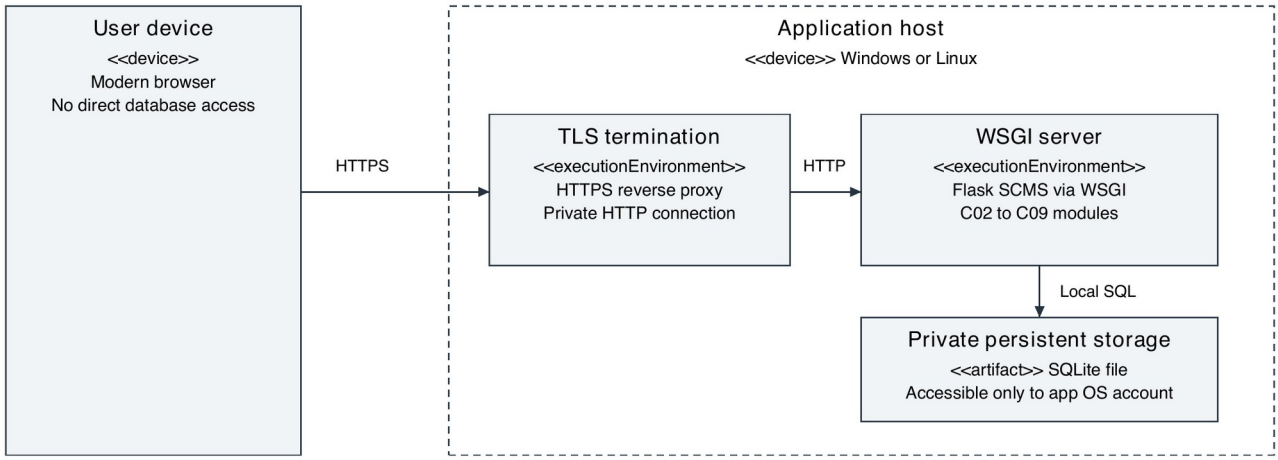
3 4 4 Manual SLA review

C08 computes elapsed time from status_updated_at using the configured threshold, default 7 days. A protected overdue view shows the current queue, days overdue and escalation availability. Escalation delegates to C04 and C05 after a fresh permission, state, version and threshold check. The scope of this exception to normal assigned-handler authority requires team review; it is not a general cross-queue handling permission.

3 5 Deployment and reliability

Deploy one SCMS application on a Windows or Linux host, with its SQLite file on durable local storage outside the web root. Use a dedicated WSGI server and HTTPS termination for shared deployment. The Flask development server is limited to local development. The browser can reach only the application HTTP interface.

AD01 SCMS Deployment and Trust Boundaries



Browser-to-host is an untrusted input boundary. SQLite and logs are never exposed through static files.

Production uses a dedicated WSGI server. Development HTTP is limited to a local test environment.

The application has one deployment unit; no email service, ERP, SSO or background SLA scheduler is required.

3 5 1 Persistence and recovery

Use short transactions and a bounded SQLite busy timeout. On lock, disk or constraint failure, roll back the entire workflow and return the Section 4 error contract. A client checks persisted state before repeating an ambiguous submission after a lost response. An SQLite-aware backup captures a consistent database; verify restoration to a separate location and normal restart persistence before release.

3 5 2 Operations and growth

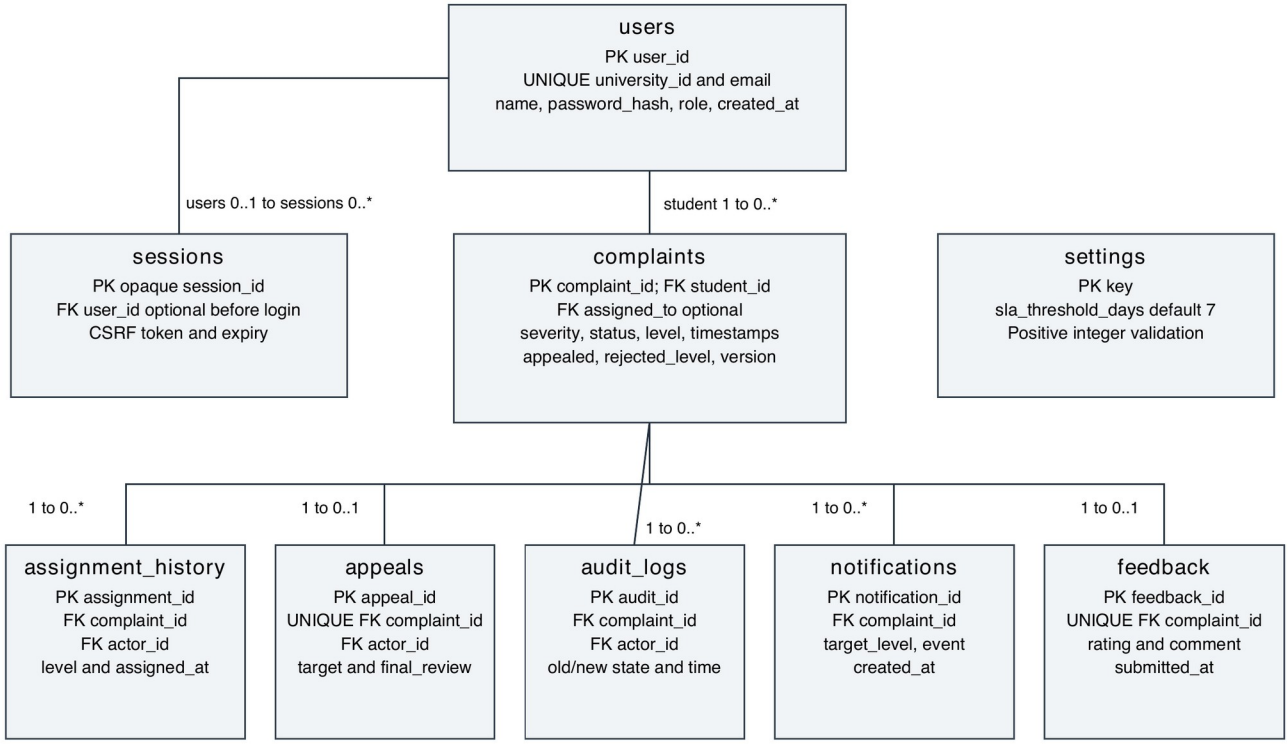
One host remains a single point of availability. Protect the database, backup, secrets and operational logs using OS permissions. Observe request latency, denied actions, failed transactions and database contention without recording sensitive request content. Pagination and targeted indexes support the 2-second dashboard and 3-second submission targets; measure these under an agreed workload rather than assuming them.

Revisit PostgreSQL and deployment capacity if concurrent writes or recovery needs exceed SQLite on one host. This is an extension path, not a current dependency. Review retention and backup frequency with the university before deployment; the SRS supplies no quantitative retention or availability target.

3 6 Logical data model

Figure AE01 identifies the persistent entities and cardinalities. users and complaints provide the core domain. Assignment, audit, appeal and notification records preserve workflow history, while sessions and settings support authentication and manual SLA review.

AE01 SCMS Logical Data Model



Cardinalities describe one parent record and its children. Every complaint has exactly one submitting user.
users also supplies actor_id for assignment, appeal and audit records; assigned_to is an optional named handler.
settings is independent. Feedback has no student_id column; authorized handler responses omit student identity.

3 6 1 Relationship and privacy rules

A complaint belongs to one student. An optional assigned_to value names a handler; current_level determines the handler queue. Each complaint has zero or one appeal and zero or one feedback record, and may have many assignment, audit and notification records. assignment_history, appeals and audit_logs each retain an actor_id referencing users. sessions.user_id is null only for a pre-login CSRF session.

Feedback stores the complaint link but no student_id. The handler-facing projection omits owner name, ID and email and never joins them into the feedback response. This satisfies identity suppression in REQ-9.2; a complaint link can still provide context, so this design does not promise unlinkability from complaint history.

3 7 Core database schema

SQLite INTEGER primary keys identify persistent records. Use TEXT for strings and UTC ISO 8601 timestamps, and INTEGER with CHECK IN (0,1) for booleans. All mandatory SRS fields use NOT NULL. Foreign keys are enabled on every connection. Field lengths below preserve SRS Appendix B. Supporting state fields extend the schema to enforce Section 4 workflows.

3 7 1 Users

Field	Type and constraint	Purpose
user_id	INTEGER PRIMARY KEY	User identity for domain relations.
university_id	TEXT NOT NULL UNIQUE; length 1-20	University account identifier.
name; email	TEXT NOT NULL; 1-100 each; email UNIQUE	Profile and contact address.
password_hash	TEXT NOT NULL; length <=255	Salted adaptive hash encoding.
role	TEXT NOT NULL; enum Student/Faculty/Committee/Dean	Stored values map to SRS user classes.
created_at	TEXT NOT NULL UTC timestamp	Creation time.

3 7 2 Complaints

Field	Type and constraint	Purpose
complaint_id; student_id	INTEGER PK; INTEGER NOT NULL FK users	Unique record and submitting owner.
category; description	TEXT NOT NULL; 1-50 and 1-1000	Validated complaint content.
severity; status	TEXT NOT NULL; constrained enumerations	Low/Medium/High and Section 3 4 states.
current_level; assigned_to	INTEGER NOT NULL CHECK 1-3; optional FK users	Role queue and optional named handler.
submitted_at; status_updated_at	TEXT NOT NULL UTC timestamps	Submission and current-state start time.
rejection_reason; resolution_remarks	TEXT nullable; maximum 500 each	Required when rejected/resolved.
appealed; rejected_level	Boolean default 0; nullable INTEGER CHECK 1-3	Single-appeal flag and rejecting queue.
final_review; version	Boolean default 0; INTEGER >=1 default 1	Dean review and concurrency control.

3 7 3 Audit logs

audit_id INTEGER PK; complaint_id and actor_id required FK; action TEXT up to 50; previous_status and new_status TEXT up to 30; timestamp required UTC TEXT. previous_level and new_level record queue changes. Use an explicit initial-state sentinel within the 30-character limit for submission history. Reject UPDATE/DELETE through application routes and database triggers; privileged database-owner access is a separate trust boundary.

3 8 Supporting schema and consistency

Entity	Keys and fields	Constraints and use
sessions	Opaque random session_id PK; optional user_id FK; CSRF token; expires_at.	Rotate on login; remove at logout/expiry. Anonymous sessions carry no authenticated role.
assignment_history	assignment_id INTEGER PK; complaint_id and actor_id FK; level; assigned_at.	Level CHECK 1-3. Records all reached queues for historic visibility.
appeals	appeal_id INTEGER PK; complaint_id UNIQUE FK; actor_id FK; justification; submitted_at; rejected_level; target_level; final_review.	One appeal per complaint; levels 1-3. Justification 1-1000 is a proposed design limit.
notifications	notification_id INTEGER PK; complaint_id FK; target_level; event; created_at.	Role queue notice stored with assignment. Access to complaint is checked when opened.
feedback	feedback_id INTEGER PK; complaint_id UNIQUE FK; rating; optional comment; submitted_at.	Rating INTEGER CHECK 1-5; comment <=500; resolved-owner eligibility enforced before write.
settings	key TEXT PK; value TEXT.	sla_threshold_days is validated as a positive integer, default 7; configured by trusted setup.

3 8 1 Transaction boundary and concurrency

A mutation uses one Repository connection for complaint state, assignment history, audit and notifications. Check the actor, legal transition and expected version within that transaction, then update WHERE complaint_id=? AND version=? and require exactly one affected row. Increment version only after a valid transition. Appeal eligibility and its unique insert are in the same transaction. Any failed write rolls back all associated changes.

3 8 2 Referential integrity and indexing

Use foreign-key restrictions rather than cascading deletion for complaint history. No complaint/audit deletion UI is specified. Parameterize values; validate and allowlist sort/filter options. Index complaints(student_id, submitted_at), complaints(current_level, status), complaints(status, status_updated_at), assignment_history(complaint_id, level), audit_logs(complaint_id, timestamp, audit_id) and notifications(target_level, created_at). Unique indexes enforce account, appeal and feedback constraints.

current_level maps 1 to Faculty, 2 to Committee and 3 to Dean. assigned_to is optional and must match that level if supplied. Historic assignment affects reads only. Database constraints, application transition rules and server-side authorization work together; foreign keys alone cannot enforce role or workflow eligibility.

3 9 Security architecture

SO-01 requires authenticated, authorized access to accounts and complaints. SO-02 requires integrity and accountability of complaint processing. C03 enforces both role and resource scope; C04 commits state and audit together. The public browser boundary accepts untrusted input, and the database and operational files remain private to the host.

3 9 1 Identity sessions and provisioning

Public registration creates only Student accounts. Trusted deployment setup provisions Faculty, Committee and Dean accounts and SLA permission. Store a salted adaptive password hash using a maintained library such as Werkzeug; select an encoding that fits the 255-character field. Verify with the corresponding library helper. Use generic failed-login messages and a bounded login throttle as an additional design safeguard.

A signed Secure, HttpOnly, SameSite=Lax cookie carries an opaque session ID; persistent session records establish the actor and current role. Rotate session ID and CSRF token on login, expire sessions and revoke on logout. Verify CSRF on mutations, including login and registration. Keep signing secrets outside source control; use HTTPS for shared deployment and disable debug mode.

3 9 2 Access control and threat handling

Threat	Architectural control	Requirement
Wrong-role or cross-owner access	C03 verifies identity, ownership, assignment and permitted action before every service call and read.	SEC-02; REQ-1.3; REQ-4.4
Password disclosure or account enumeration	Salted adaptive hashes; generic failures; no hashes/passwords in responses or logs.	SEC-01; SEC-04
SQL injection or stored script content	C09 parameterizes SQL; allowlist query structure; escape user text in C02 output.	SO-01; SO-02
CSRF or stolen session token	CSRF verification, secure cookie attributes, HTTPS and session revocation.	SO-01; additional safeguards
Audit edits or partial workflow state	Append-only C06; no mutation route; trigger protection; atomic C04 transaction.	REQ-6.1 to REQ-6.3; SEC-03
Duplicate appeal or stale mutation	Unique appeal and feedback constraints plus optimistic version checking.	REQ-5.1 to REQ-5.4; SO-02

3 9 3 Privacy and operational access

Return minimum role-specific fields and apply Cache-Control: no-store to protected data. Denied access to an invisible complaint uses a generic not-found result. Logs exclude complaint text, appeal justification, feedback comments, passwords, hashes, cookies and CSRF tokens. Authorized handler feedback views exclude student identity. Database owner, backup and log administrators remain privileged; database audit triggers do not defend against host compromise.